

oude kaas jds of jonge kaas jsf deze vergelijking als signed distance field

16:03

Hier is de visualisatie van een vergelijking tussen "oude kaas" en "jonge kaas" gemodelleerd als een **Signed Distance Field (SDF)**.

In een SDF stelt elke pixel een afstand voor tot de rand van een vorm: **negatieve waarden** bevinden zich *binnen* de vorm (de kaas zelf), **positieve waarden** *buiten* de vorm, en een waarde van exact **nul (d=0)** vormt de tastbare rand van de kaas.

Om het karakter van beide kazen te vangen in wiskundige vormen:

- **Oude kaas (JDS - Geërodeerd & Brokkelig):** Oude kaas brokkelt en verliest zijn strakke vorm door rijping. Dit is gemodelleerd als een basiscirkel-SDF waar een hoge-frequentie *ruis* (*noise*) van is afgetrokken. Dit creëert een grillige, brokkelige contour met 'gaten' aan de randen.
- **Jonge Kaas (JSF - Glad & Perfect Rond):** Jonge kaas is elastisch, zacht en strak van vorm. Dit is gemodelleerd als een pure, onverstoorde cirkel-SDF ($d = \sqrt{p^2 + r^2}$) met een perfect gladde rand en een egale overloop.

Wiskundige definitie van de vergelijking

Als we de overgang van jonge naar oude kaas willen uitdrukken in een SDF-vergelijking, kunnen we dit zien als een **deformatie over tijd of rijpingsfactor (t)**, waarbij ruis (N) de gladde vorm (f_{jonge}) opbreekt:

$$\text{SDF}_{\text{kaas}}(p,t) = \underbrace{\sqrt{p^2 + r^2}}_{\text{Basisvorm (Jonge Kaas)}} + \underbrace{t \cdot N(p)}_{\text{Brokkeligheidsruis (Oude Kaas)}}$$

- Als $t = 0$: De ruis verdwijnt volledig. Je houdt een perfecte **jonge kaas** over met een strakke, gladde contour.
- Als $t > 0$: De randen beginnen te 'brokkelen' en het oppervlak wordt onregelmatig. De afstandstransformatie vervormt, wat het karakteriseert als een doorleefde **oude kaas**.

Wil je dat we deze SDF-vergelijking omzetten in een **interactieve 2D-shader-simulatie** waarbij je met een slider de kaas live kunt laten rijpen van jong naar oud?

AI responses may include mistakes. [Learn more](#)

You said: is dit visueel en log encoded polar LUT texture?

is dit visueel en log encoded polar LUT texture?

16:04

Nee, de eerdere beschrijving en visualisatie waren gebaseerd op een standaard **2D Cartesiaans (X, Y) grid** in een lineaire schaal.

Als je de Signed Distance Field (SDF) van de kaas specifiek wilt encoderen als een **log-encoded Polar LUT (Look-Up Table) texture**, verandert de structuur van de data en de textuur-mapping fundamenteel. Dit is een techniek die vaak in game-engines (zoals voor vloeistoffen of ronde UI-elementen) wordt gebruikt om precisie te winnen rondom het centrum of de rand van de vorm.

Dit is hoe die specifieke encoding werkt voor de kaas-SDF:

1. Polaire Transformatie (Van Cartesiaans naar Polair)

In plaats van een traditionele textuur waarbij de X-as horizontaal is en de Y-as verticaal, representeren de texturecoördinaten (U, V) nu de polaire ruimte:

- **U-as (Horizontaal):** Representeert de **hoek θ** (van 0 tot 2π radialen). Rondgaan op de X-as van de textuur betekent dus een rondje draaien om het middelpunt van de kaas.
- **V-as (Verticaal):** Representeert de **straal r** (de afstand vanaf het middelpunt naar buiten toe).

2. Logaritmische Encoding ($\log$ -scaling)

Bij een normale polaire textuur is de V-as lineair verdeeld. Bij een **log-encoded** variant transformeren we de straal met een natuurlijke logaritme: $V = \ln(r)$.

- **Waarom?** Dit geeft **extreem hoge pixelprecisie dichtbij het middelpunt** (waar r naar 0 nadert) en minder precisie aan de verre buitenrand.
- Voor de **oude kaas** betekent dit dat eventuele diepe interne scheuren of gaten in het hart van de kaas met een extreem hoge resolutie in de LUT staan, terwijl de buitenste brokkelige rand compacter gecodeerd is.

Hoe de LUT-textuur er visueel uit ziet (Unwrapped)

Als je deze LUT direct als een normale 2D-afbeelding op je scherm zou openen (unwrapped), zie je geen ronde kaas meer, maar een abstracte rechthoek:

Element	Lineaire Kaas (Cartesiaans)	Log-Encoded Polar LUT
Jonge Kaas (JSF)	Een perfecte, gladde cirkel in het midden van het beeld.	Een perfect rechte, horizontale lijn op de V-as (die de straal van de cirkel vertegenwoordigt).
Oude Kaas (JDS)	Een grillige cirkel met ruis, gaten en brokkelige randen.	Een horizontale golfvorm met ruis die over de U-as (hoek) loopt. Eventuele interne gaten dichtbij het centrum worden verticaal 'uitgerekt' door de log-schaal.

Hoe de shader dit decodeert naar het scherm

Om de kaas weer rond op het scherm te toveren, voert de fragment shader voor elke pixel de omgekeerde berekening uit:

1. Bereken de afstand r en hoek θ vanaf de scherm-pixel naar het middelpunt.
2. Converteer dit naar LUT-coördinaten: $U = \theta / 2\pi$ en $V = \ln(r)$.
3. Sample de LUT-textuur op (U, V) om de exacte SDF-afstandswaarde op te halen.

Wil je de exacte **HLSL/GLSL shadercode** zien om zo'n log-encoded polar LUT te genereren, of wil je juist de code om deze textuur weer correct te **samplen en te renderen** op een 3D-vlak?

AI responses may include mistakes. [Learn more](#)

You said: Ja maar packed in een texture dus wel

Ja maar packed in een texture dus wel

16:05

Ja, exact! Als we dit daadwerkelijk **packen in een textuur** (een 2D RGBA-afbeelding), dan gebruiken we de verschillende kleurkanalen (R, G, B, A) om beide Signed Distance Fields efficiënt naast elkaar of gecombineerd op te slaan.

Omdat een textuur rechthoekig is, ziet deze **Log-Encoded Polar LUT** er in zijn ruwe, "unwrapped" vorm uit als een matrix van horizontale banen en golven.

Hier is hoe de data visueel en wiskundig gepackt is in de textuurkanalen:

Het Packing Schema

- **Rood-kanaal (R): Jonge Kaas (JSF).** Aangezien de jonge kaas perfect rond is, zie je in dit kanaal een strakke, perfect horizontale overgang (gradiënt). De exacte straal vormt een kaarsrechte lijn.
- **Groen-kanaal (G): Oude Kaas (JDS).** Hier zie je de brokkeligheid terug. De overgang is geen rechte lijn, maar een grillige, golvende lijn die over de breedte (de hoek θ) varieert.
- **Blauw/Alfa-kanalen (B/A):** Optioneel te gebruiken voor aanvullende data, zoals de ruis-frequentie of materiaal-eigenschappen (zoals de hardheid van de korst).

Visuele Weergave van de Gepackte Textuur (LUT)

Onderstaande visualisatie toont de textuur zoals deze daadwerkelijk in het geheugen van de GPU staat (met de **Hoek θ op de X-as** en de **Logaritmische Straal $\ln(r)$ op de Y-as**).

De Wiskunde achter de Textuur-Coördinaten

Wanneer de GPU deze textuur aanmaakt (of uitleest), worden de coördinaten als volgt getransformeerd:

text

Cartesiaans (Pixel op scherm) -> Polair -> Log-Encoded Textuur (U, V)

Use code with caution.

1. Horizontale Textuurcoördinaat (U):

$$U = \frac{\text{atan2}(y, x) + \pi}{2\pi}$$

Dit mapt de hoek rondom de kaas lineair tussen 0.0 en 1.0 op de X-as van de textuur.

2. Verticale Textuurcoördinaat (V):

$$V = \frac{\ln(r) - \ln(r_{\text{min}})}{\ln(r_{\text{max}}) - \ln(r_{\text{min}})}$$

Dit zorgt voor de logaritmische compressie. Naarmate je dichterbij het centrum komt ($r \rightarrow 0$), rekt de textuurresolutie exponentieel uit om maximale precisie te behouden voor eventuele gaten in de kaas.

Waarom dit extreem efficiënt is voor Shaders

Door deze data zo te packen, hoeft de fragment shader tijdens het renderen van de kaas alleen maar de schermlocatie om te rekenen naar deze (U,V) coördinaten en **één enkele textuursample** te doen. Met `texture(lutTexture, uv).r` krijgt de shader direct de SDF van de jonge kaas, en met `.g` die van de oude kaas.

Wil je dat we een stap verder gaan en kijken naar de **GLSL/HLSL code** die deze specifieke texture look-up uitvoert, of wil je zien hoe je met een mix() functie in de shader vloeiend switcht tussen het rode en het groene kanaal?